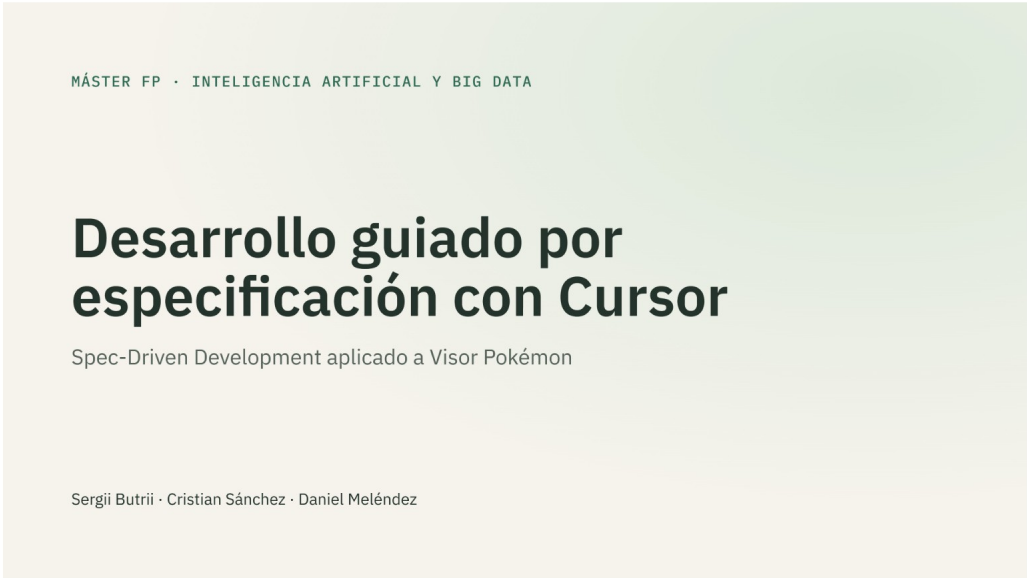
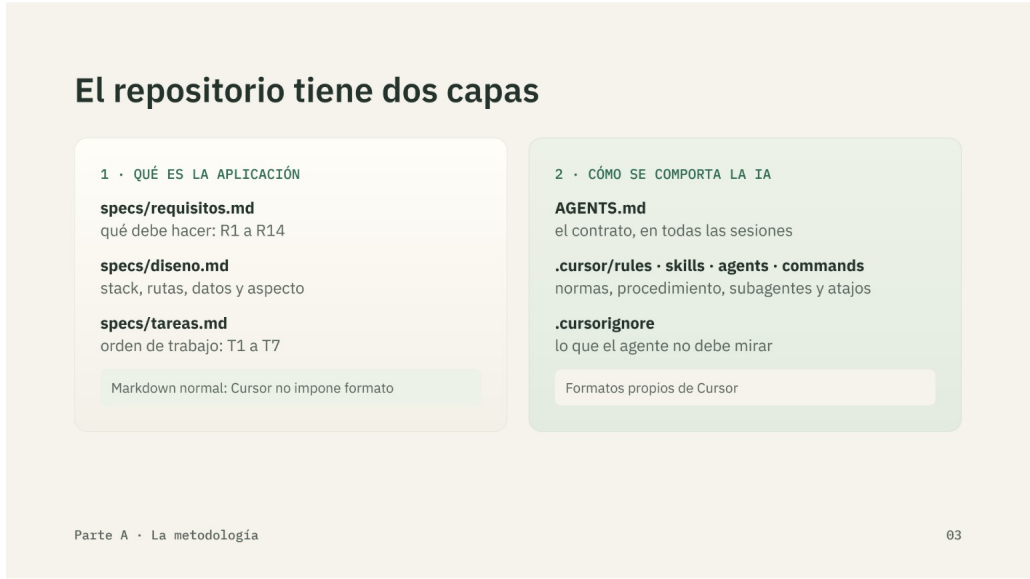


Buenos días. Somos Sergii, Cristian y Daniel. Vamos a presentar el desarrollo guiado por especificación, SDD, con Cursor como herramienta, aplicado a una aplicación sencilla: Visor Pokémon. Primero explico la metodología y después nuestra experiencia práctica.



SDD es Spec-Driven Development: desarrollo guiado por especificación. Lo resume esta frase de nuestro AGENTS.md: la especificación manda y el código la sirve. Primero se escribe qué hay que construir, después la IA lo implementa y una persona dirige y supervisa. ¿Por qué en archivos? Porque el agente no recuerda la conversación anterior. Lo que queda en un chat se pierde; lo que está en el repositorio lo leen todo el equipo y cada sesión del agente.



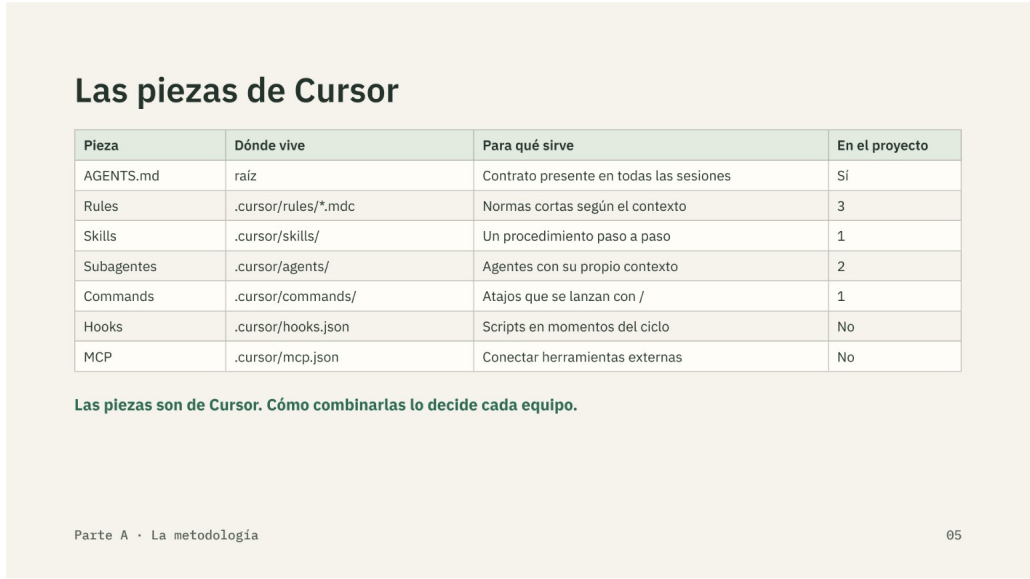
El repositorio tiene dos capas. La primera es la especificación, en la carpeta specs: requisitos, con criterios comprobables del R1 al R14; diseño, con el stack, las rutas y el aspecto; y tareas, con el orden de trabajo de la T1 a la T7. Es Markdown normal: Cursor no impone formato. La segunda capa es la configuración del agente: AGENTS.md y la carpeta .cursor. No describe la aplicación: obliga a la IA a leer la especificación y a no salirse de ella.

Fases del proceso

#	Fase	Dónde	Qué se hace
1	Requisitos	requisitos.md	Alcance y criterios comprobables
2	Diseño	diseno.md	Pantallas, datos, archivos y aspecto
3	Tareas	tareas.md	Pasos pequeños, en orden
4	Implementación	/implementar-tarea	La IA programa una sola tarea
5	Revisión	revisor-spec	Cumple o no cumple, criterio a criterio

Cambio de requisitos: se vuelve a la fase 1. Primero la spec, después el código.

El proceso tiene cinco fases, y el orden se evalúa: no hay código hasta que las specs están en el repositorio. Requisitos: alcance y criterios que se comprueban con un sí o un no. Diseño: pantallas, datos y aspecto, para que la IA no los cambie a mitad de tarea. Tareas: pasos pequeños y ordenados. Implementación: un comando, una tarea. Revisión: criterio a criterio. Si cambian los requisitos, se vuelve a la fase uno: primero la spec, después el código.



Las piezas de Cursor

Pieza	Dónde vive	Para qué sirve	En el proyecto
AGENTS.md	raíz	Contrato presente en todas las sesiones	Sí
Rules	.cursor/rules/*.mdc	Normas cortas según el contexto	3
Skills	.cursor/skills/	Un procedimiento paso a paso	1
Subagentes	.cursor/agents/	Agentes con su propio contexto	2
Commands	.cursor/commands/	Atajos que se lanzan con /	1
Hooks	.cursor/hooks.json	Scripts en momentos del ciclo	No
MCP	.cursor/mcp.json	Conectar herramientas externas	No

Las piezas son de Cursor. Cómo combinarlas lo decide cada equipo.

Parte A · La metodología 05

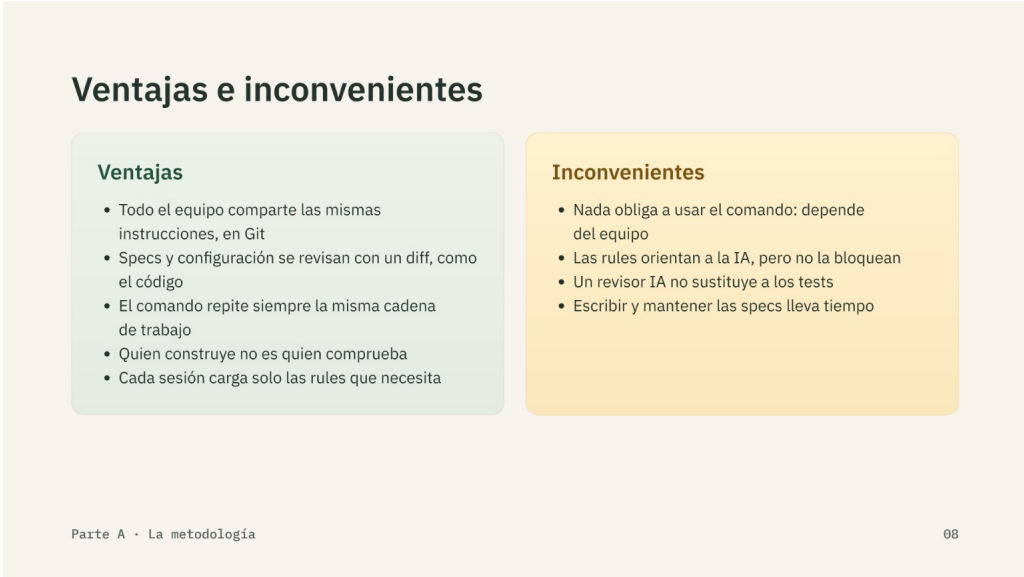
Cursor no tiene una forma oficial de hacer SDD: ofrece piezas para configurar el agente. AGENTS.md es el contrato presente en todas las sesiones. Las rules son normas cortas que se cargan según el archivo que se toca. La skill describe un procedimiento. Los subagentes tienen su propio contexto y reciben trabajo delegado. Los commands son atajos que se lanzan con barra. Hooks y MCP existen, pero no los necesitamos. Las piezas son de Cursor; cómo combinarlas lo decide cada equipo.



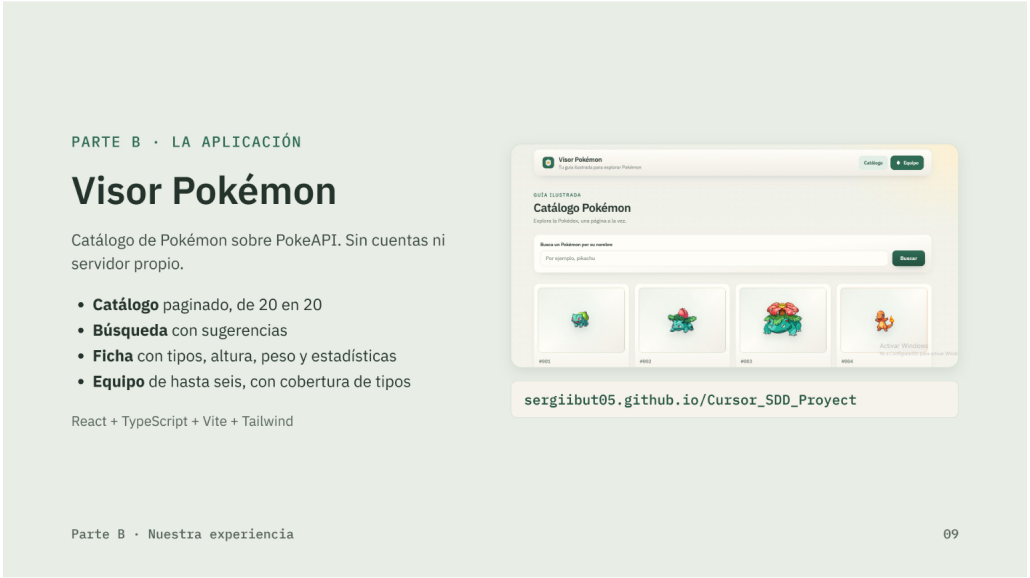
Así se usa. La persona escribe barra implementar-tarea y el número, por ejemplo T3. El comando delega en el subagente implementador, que sigue la skill: lee la tarea, el diseño y los criterios, y programa solo esa tarea. Después, el subagente revisor-spec, que no puede editar, dice qué criterios se cumplen y cuáles no. Al final decide la persona: corrige el código o, si el fallo está en la especificación, la cambia primero. Mientras tanto, las rules se cargan solas.



Comparado con el ejercicio de clase con Copilot, el método es el mismo: primero la especificación y después el código. Cambia dónde está la disciplina. En clase pedíamos los archivos uno a uno y éramos nosotros quienes exigíamos seguir la spec. Con Cursor, esa exigencia está escrita en archivos que el agente carga solo, y un subagente revisa. Pero la persona sigue revisando: el revisor ayuda, no sustituye.



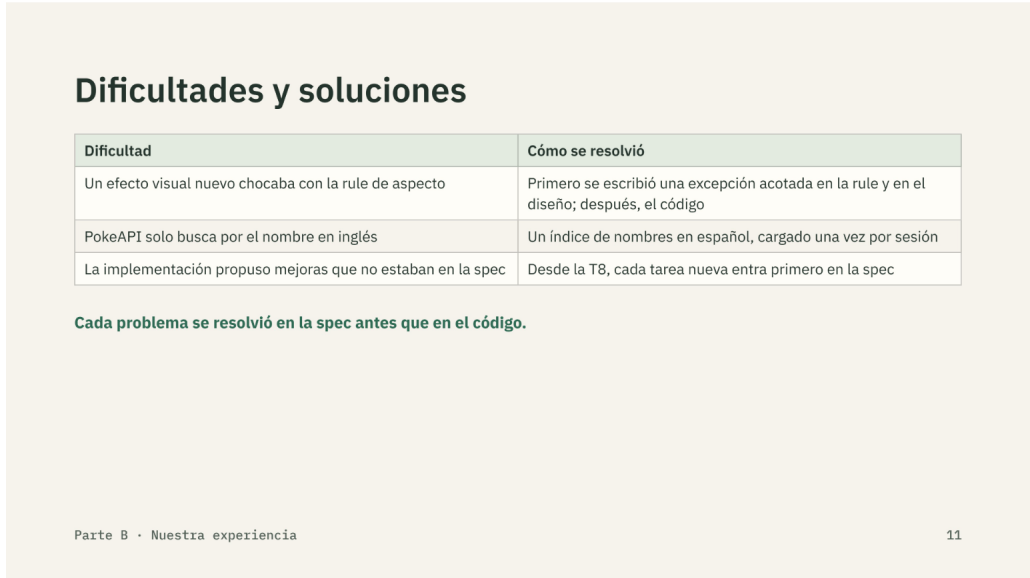
Ventajas: las instrucciones están en Git y todo el equipo usa las mismas; la spec se revisa con un diff, como el código; el comando repite siempre el mismo proceso; quien programa no es quien comprueba; y las rules se cargan solo cuando hacen falta. Inconvenientes: nada impide pedir código ignorando la spec, así que depende del equipo; las rules orientan pero no bloquean; un revisor IA no sustituye a los tests; y mantener las specs lleva tiempo.



Pasamos a la Parte B: nuestra experiencia. La aplicación es Visor Pokémon, un catálogo que consulta PokeAPI, sin cuentas y sin servidor propio. Tiene cuatro funcionalidades: un catálogo paginado de veinte en veinte, búsqueda por nombre, una ficha con tipos, altura, peso y estadísticas, y un apartado extra para guardar un equipo de hasta seis Pokémon, con un panel que muestra la cobertura de tipos. Está hecha con React, TypeScript, Vite y Tailwind. Está publicada en GitHub Pages; os la enseño en directo. DEMO, 2 minutos, con la web abierta en otra pestaña: 1) Catálogo: bajar y pulsar Cargar más. 2) Escribir pi en el buscador: salen sugerencias, es el cambio C1. 3) Abrir Pikachu: 0,4 metros, 6 kilos, PS 35, justo lo que pide el requisito R7. 4) Añadir al equipo, ir a Equipo y enseñar Golpea con ventaja y Recibe daño doble. 5) Pulsar un tipo de Recibe daño doble: catálogo filtrado y Quitar filtro, también C1. Si falla internet, se queda esta captura y se explica sobre ella.



Así lo aplicamos. El 23 de octubre se escribieron las especificaciones y la configuración de Cursor, antes de cualquier línea de código. Ese mismo día cambiamos el estilo visual, y lo hicimos en la especificación y en la rule de aspecto, no en el código. El 25 se subió la implementación de las tareas T1 a T7 y el README con los pasos para ejecutar. El 26 se añadieron tres tareas nuevas a la spec: la T8, publicar la aplicación en GitHub Pages; la T9, el cambio de requisitos; y la T10, un efecto holográfico en las tarjetas, escrito primero en el diseño y en la rule de aspecto. Para verificar el resultado: los 35 tests automáticos pasan, los textos exactos que piden los requisitos están en el código y el proyecto compila sin errores.



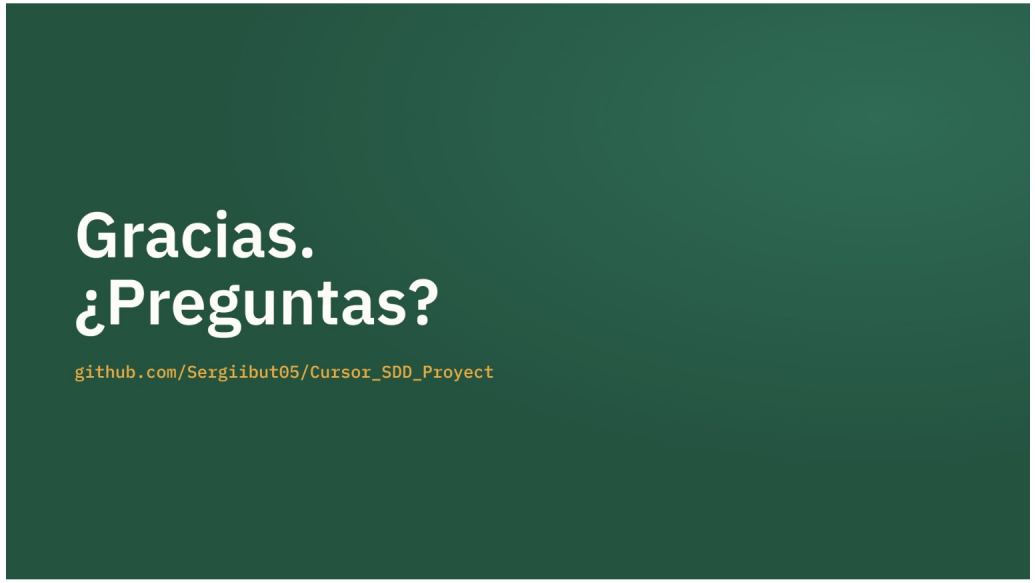
Tres dificultades. Primera: queríamos un efecto holográfico en las tarjetas, pero la rule de aspecto prohibía degradados llamativos. En vez de saltársela, se escribió primero una excepción acotada en la rule y en el diseño, y solo después el código. Segunda: PokeAPI solo busca por el nombre en inglés, así que para buscar en español se creó un índice de nombres que se carga una vez por sesión. Tercera: al implementar surgieron mejoras que no estaban en la spec; desde la T8, cada tarea nueva se escribe primero en la spec. En los tres casos, el problema se resolvió en la especificación antes que en el código.



El cambio de requisitos estaba escrito desde el primer día como C1, reservado hasta cerrar el MVP. Antes, la búsqueda solo funcionaba con el nombre exacto en inglés. Ahora muestra sugerencias mientras escribes, también con el nombre en español. Y cada debilidad del equipo filtra el catálogo con los Pokémon que la cubren. Se gestionó con el método: se actualizaron los requisitos, el diseño y la rule de PokeAPI, se creó la tarea T9, se implementó y se añadieron tests. Los 26 pasan.



Nuestra valoración. Lo que funcionó: con una especificación tan concreta, la IA cumplió los criterios; los textos y valores que pedían los requisitos están en el código. Los criterios exactos, como los datos de Pikachu, se pueden convertir directamente en tests, y ya hay 35. Y hasta los cambios visuales, como el efecto holográfico, pasaron primero por la spec. Lo que mejoraríamos: guardar registro del uso del agente, con una bitácora y capturas, para poder enseñar el proceso; escribir la accesibilidad y el uso en móvil como requisitos comprobables, no solo en el diseño; y probar un hook de Cursor que avise si el código se adelanta a la especificación. Conclusión: la IA cumple lo que está escrito; la calidad depende de cómo escribimos la spec.



Muchas gracias. Todo lo que he contado está en el repositorio: las especificaciones, la configuración de Cursor, la bitácora y esta presentación. Si tenéis preguntas, adelante.